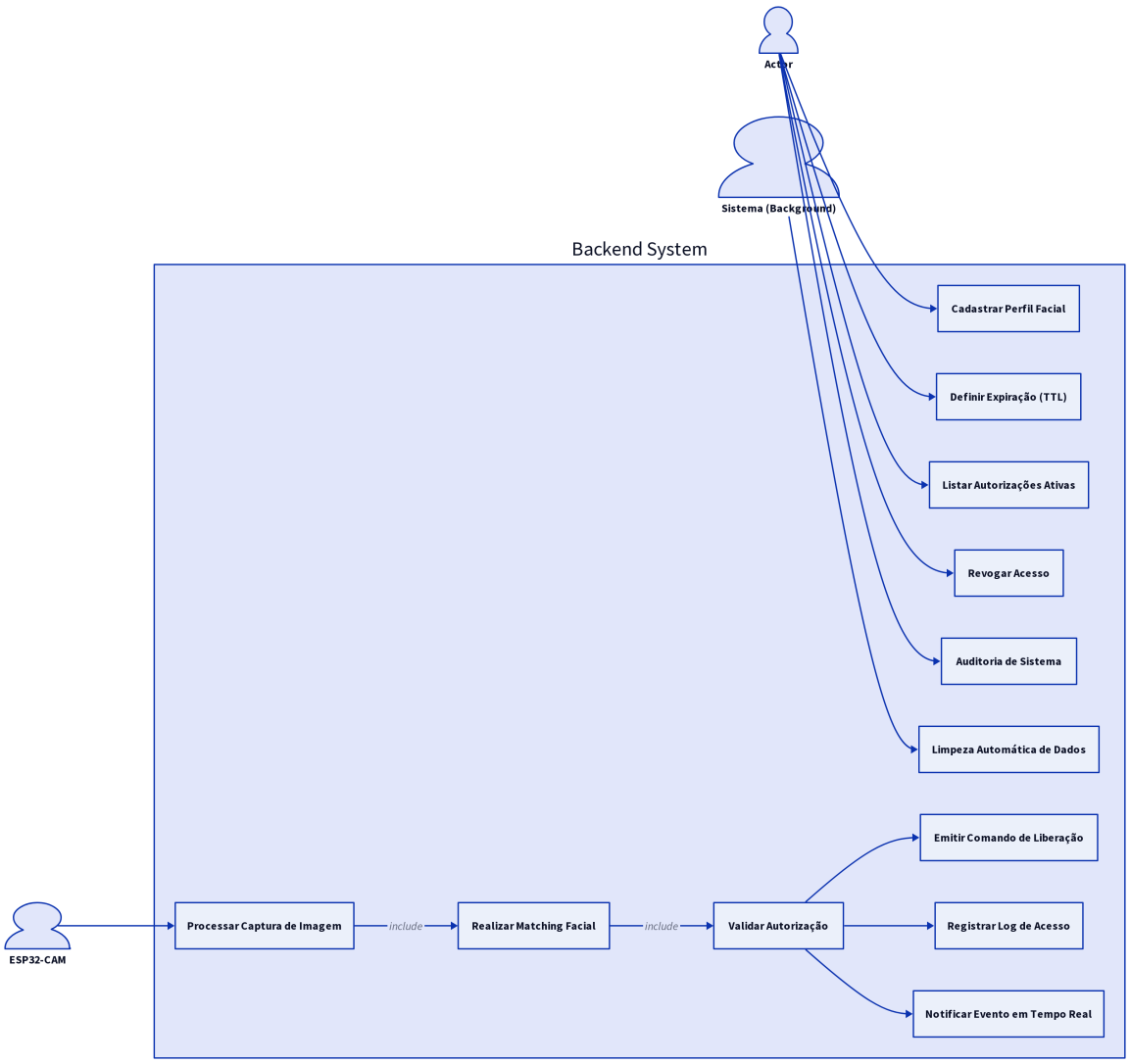
Documentação do Backend: Sistema de Reconhecimento Facial

Este documento detalha o funcionamento do backend para o sistema de controle de acesso baseado em reconhecimento facial, incluindo casos de uso e a estrutura do banco de dados.

---

1. Diagrama de Casos de Uso

O diagrama abaixo ilustra as interações entre os atores (Morador, ESP32-CAM e Sistema) e as funcionalidades do backend.



Descrição das Funcionalidades

1.1 Gestão de Identidade e Acessos (Ator: Morador)

- **Cadastrar Perfil Facial**: O backend recebe a imagem, detecta o rosto, extrai o embedding (vetor) e armazena no banco vetorial.
- **Definir Expiração (TTL)**: Atribuir um tempo de validade para um acesso específico.
- **Listar Autorizações Ativas**: Consultar no banco quais perfis ainda são válidos.
- **Revogar Acesso**: Remover manualmente um embedding do banco de dados antes da expiração.

1.2 Processamento de Reconhecimento (Ator: ESP32-CAM)

- **Processar Captura de Imagem**: Receber o frame enviado pelo hardware após o gatilho do sensor ultrassônico.
- **Realizar Matching Facial**: Comparar o embedding da imagem recebida com os vetores armazenados usando distância de cosseno.
- **Validar Autorização**: Verificar se o rosto reconhecido possui uma permissão ativa e dentro do prazo de validade.

1.3 Controle e Resposta (Fluxo de Descida)

- **Emitir Comando de Liberação**: Enviar sinal de sucesso para o ESP32-CAM ativar o relé/LED.
- **Registrar Log de Acesso**: Salvar histórico de tentativas (sucesso ou falha) com data, hora e foto capturada.
- **Notificar Evento em Tempo Real**: Disparar um alerta via WebSocket para o aplicativo do morador informando quem está no local.

1.4 Manutenção e Segurança

- **Limpeza Automática de Dados**: Serviço em segundo plano que remove embeddings cujos convites já expiraram.
- **Auditoria de Sistema**: Fornecer dados para o dashboard do aplicativo sobre o uso e tentativas de acesso não autorizadas.

---

2. Estrutura do Banco de Dados

O sistema utiliza um banco de dados relacional com suporte a vetores (ex: PostgreSQL com `pgvector`) para armazenar perfis e logs.

Diagrama UML do Banco de Dados